

```
In [1]: import pandas as pd
```

```
In [2]: import matplotlib.pyplot as plt
```

```
In [3]: import seaborn as sns
```

```
In [4]: from sklearn.model_selection import train_test_split
```

```
In [5]: from sklearn.linear_model import LinearRegression
```

```
In [6]: from sklearn.metrics import mean_squared_error, r2_score
```

```
In [7]: data = pd.read_csv("advertising (1).csv")
```

FileNotFoundError Traceback (most recent call last)

Cell In[7], line 1

```
----> 1 data = pd.read_csv("advertising (1).csv")
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1026, in read_csv(filepath_or_buffer, sep, delimiter, header, names, index_col, usecols, dtype, engine, converters, true_values, false_values, skip_initialspace, skiprows, skipfooter, nrows, na_values, keep_default_na, na_filter, verbose, skip_blank_lines, parse_dates, infer_datetime_format, keep_date_col, date_parser, date_format, dayfirst, cache_dates, iterator, chunksize, compression, thousands, decimal, lineterminator, quotechar, quoting, doublequote, escapechar, comment, encoding, encoding_errors, dialect, on_bad_lines, delim_whitespace, low_memory, memory_map, float_precision, storage_options, dtype_backend)

```
1013 kwds_defaults = _refine_defaults_read(
1014     dialect,
1015     delimiter,
1016     (...)
1022     dtype_backend=dtype_backend,
1023 )
1024 kwds.update(kwds_defaults)
-> 1026 return _read(filepath_or_buffer, kwds)
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:620, in _read(filepath_or_buffer, kwds)

```
617 _validate_names(kwds.get("names", None))
619 # Create the parser.
--> 620 parser = TextFileReader(filepath_or_buffer, **kwds)
622 if chunksize or iterator:
623     return parser
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1620, in TextFileReader.__init__(self, f, engine, **kwds)

```
1617 self.options["has_index_names"] = kwds["has_index_names"]
1619 self.handles: IOHandles | None = None
-> 1620 self._engine = self._make_engine(f, self.engine)
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\parsers\readers.py:1880, in TextFileReader._make_engine(self, f, engine)

```
1878 if "b" not in mode:
1879     mode += "b"
-> 1880 self.handles = get_handle(
1881     f,
1882     mode,
1883     encoding=self.options.get("encoding", None),
1884     compression=self.options.get("compression", None),
1885     memory_map=self.options.get("memory_map", False),
1886     is_text=is_text,
1887     errors=self.options.get("encoding_errors", "strict"),
1888     storage_options=self.options.get("storage_options", None),
1889 )
1890 assert self.handles is not None
1891 f = self.handles.handle
```

File C:\ProgramData\anaconda3\Lib\site-packages\pandas\io\common.py:873, in get_handle(path_or_buf, mode, encoding, compression, memory_map, is_text, errors, storage_options)

```
868 elif isinstance(handle, str):
869     # Check whether the filename is to be opened in binary mode.
870     # Binary mode does not support 'encoding' and 'newline'.
871     if ioargs.encoding and "b" not in ioargs.mode:
872         # Encoding
--> 873         handle = open(
874             handle,
875             ioargs.mode,
876             encoding=ioargs.encoding,
877             errors=errors,
878             newline="",
879         )
880     else:
881         # Binary mode
882         handle = open(handle, ioargs.mode)
```

FileNotFoundError: [Errno 2] No such file or directory: 'advertising (1).csv'

In [8]: data = pd.read_csv(r"C:\Users\neha gupta\Downloads\advertising (1).csv")

In [9]: data.head()

```
Out[9]:
```

	TV	Radio	Newspaper	Sales
0	230.1	37.8	69.2	22.1
1	44.5	39.3	45.1	10.4
2	17.2	45.9	69.3	12.0
3	151.5	41.3	58.5	16.5
4	180.8	10.8	58.4	17.9

```
In [10]: print(data.info())

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 200 entries, 0 to 199
Data columns (total 4 columns):
#   Column      Non-Null Count  Dtype  
---  -
0    TV          200 non-null    float64
1    Radio       200 non-null    float64
2    Newspaper   200 non-null    float64
3    Sales       200 non-null    float64
dtypes: float64(4)
memory usage: 6.4 KB
None
```

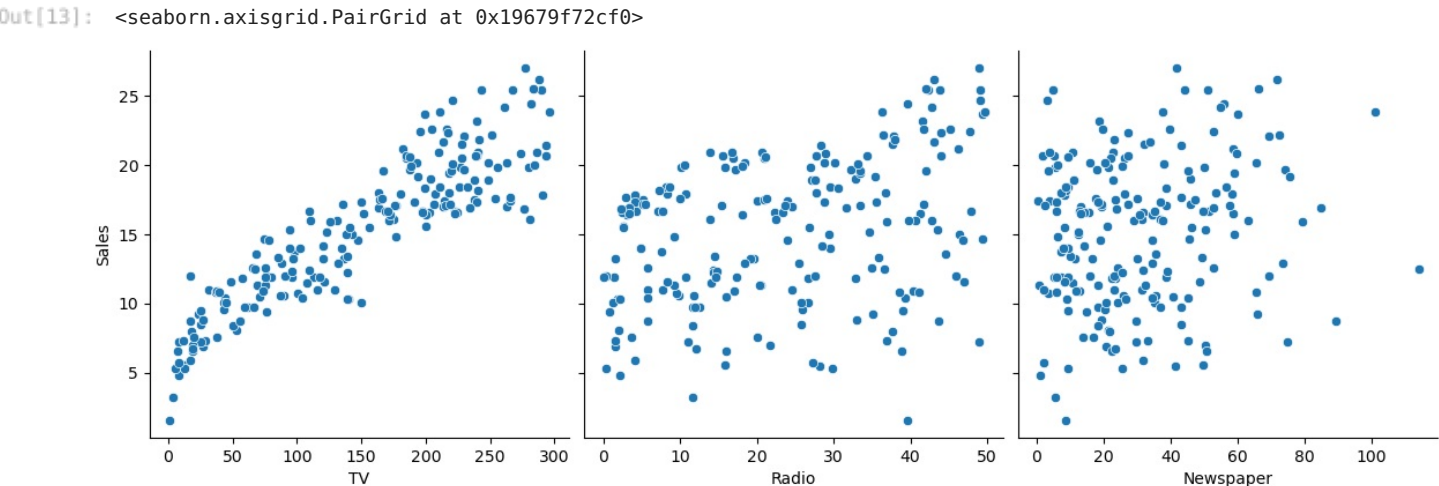
```
In [11]: print(data.describe())
```

	TV	Radio	Newspaper	Sales
count	200.000000	200.000000	200.000000	200.000000
mean	147.042500	23.264000	30.554000	15.130500
std	85.854236	14.846809	21.778621	5.283892
min	0.700000	0.000000	0.300000	1.600000
25%	74.375000	9.975000	12.750000	11.000000
50%	149.750000	22.900000	25.750000	16.000000
75%	218.825000	36.525000	45.100000	19.050000
max	296.400000	49.600000	114.000000	27.000000

```
In [12]: print(data.isnull().sum())

TV          0
Radio       0
Newspaper   0
Sales       0
dtype: int64
```

```
In [13]: sns.pairplot(data, x_vars=['TV','Radio','Newspaper'], y_vars='Sales', height=4, aspect=1, kind='scatter')
```



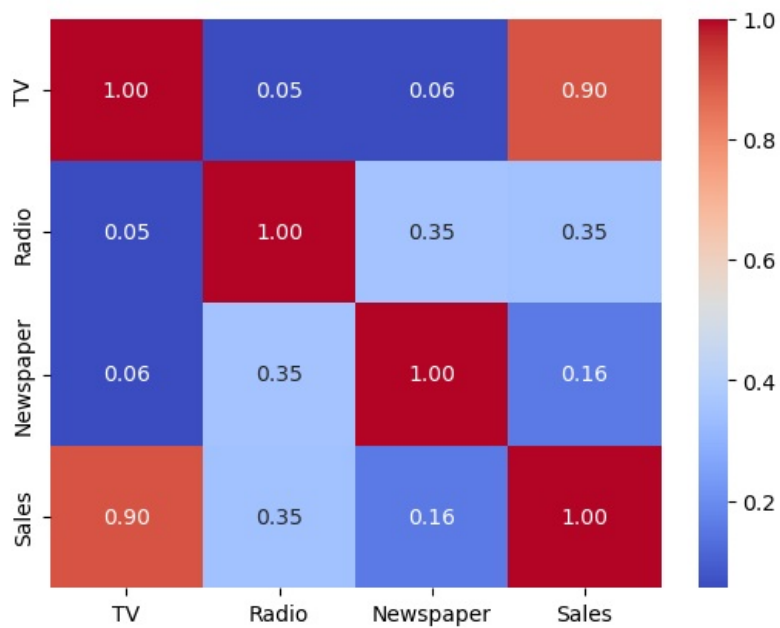
```
In [14]: plt.show()
```

```
In [15]: plt.figure(figsize=(6,4))
```

```
Out[15]: <Figure size 600x400 with 0 Axes>
<Figure size 600x400 with 0 Axes>
```

```
In [16]: sns.heatmap(data.corr(), annot=True, cmap="coolwarm", fmt=".2f")
```

```
Out[16]: <Axes: >
```



```
In [17]: plt.show()
```

```
In [18]: X = data[['TV', 'Radio', 'Newspaper']]
```

```
In [19]: y = data['Sales']
```

```
In [20]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
In [21]: model = LinearRegression()
```

```
In [22]: model.fit(X_train, y_train)
```

```
Out[22]: ▼ LinearRegression ⓘ ?  
LinearRegression()
```

```
In [23]: y_pred = model.predict(X_test)
```

```
In [24]: print("Intercept:", model.intercept_)
```

```
Intercept: 4.714126402214134
```

```
In [25]: print("Coefficients:", model.coef_)
```

```
Coefficients: [0.05450927 0.10094536 0.00433665]
```

```
In [26]: print("Mean Squared Error:", mean_squared_error(y_test, y_pred))
```

```
Mean Squared Error: 2.907756910271091
```

```
In [27]: print("R2 Score:", r2_score(y_test, y_pred))
```

```
R2 Score: 0.9059011844150826
```

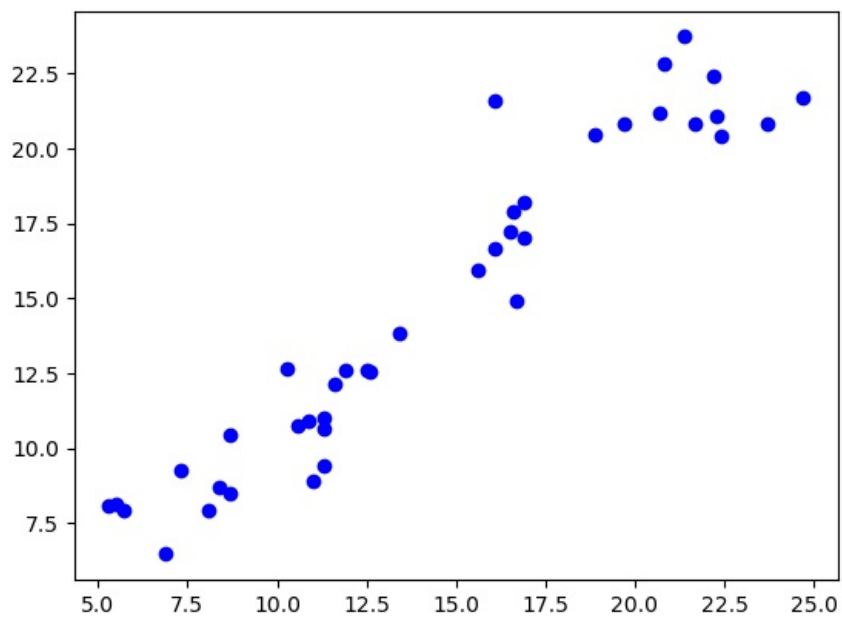
```
In [28]: plt.figure(figsize=(6,4))
```

```
Out[28]: <Figure size 600x400 with 0 Axes>
```

```
<Figure size 600x400 with 0 Axes>
```

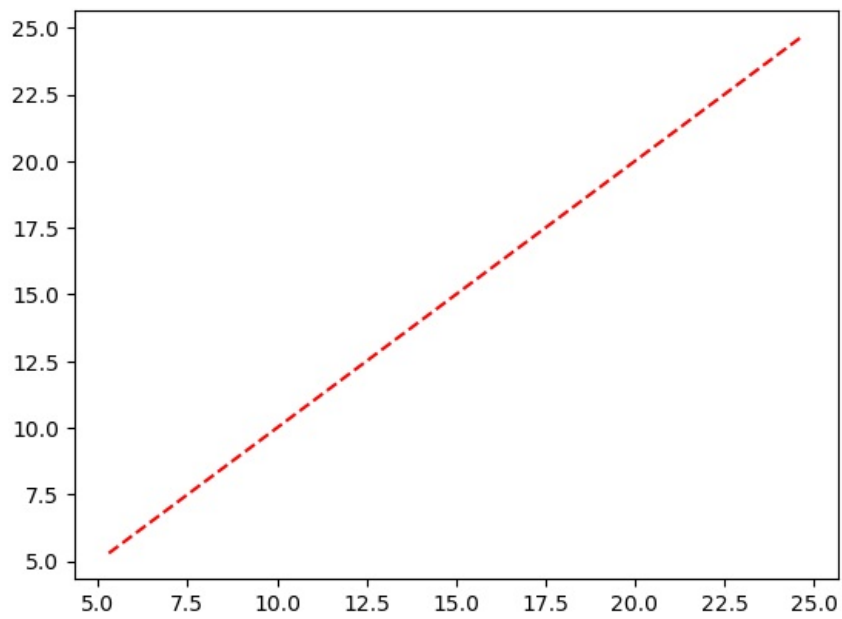
```
In [29]: plt.scatter(y_test, y_pred, color='blue')
```

```
Out[29]: <matplotlib.collections.PathCollection at 0x1967d795090>
```



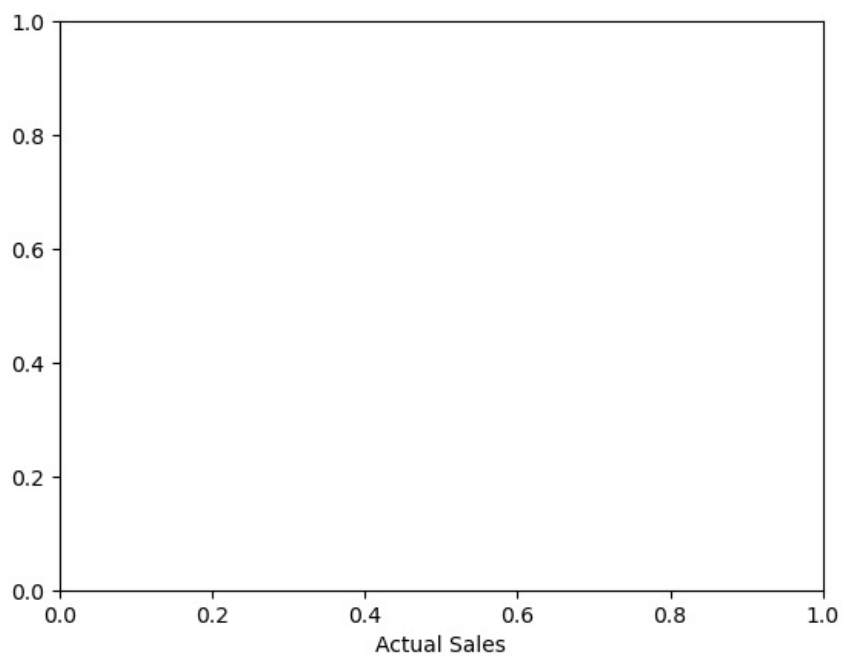
```
In [30]: plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--')
```

```
Out[30]: [<matplotlib.lines.Line2D at 0x1967d81ccd0>]
```



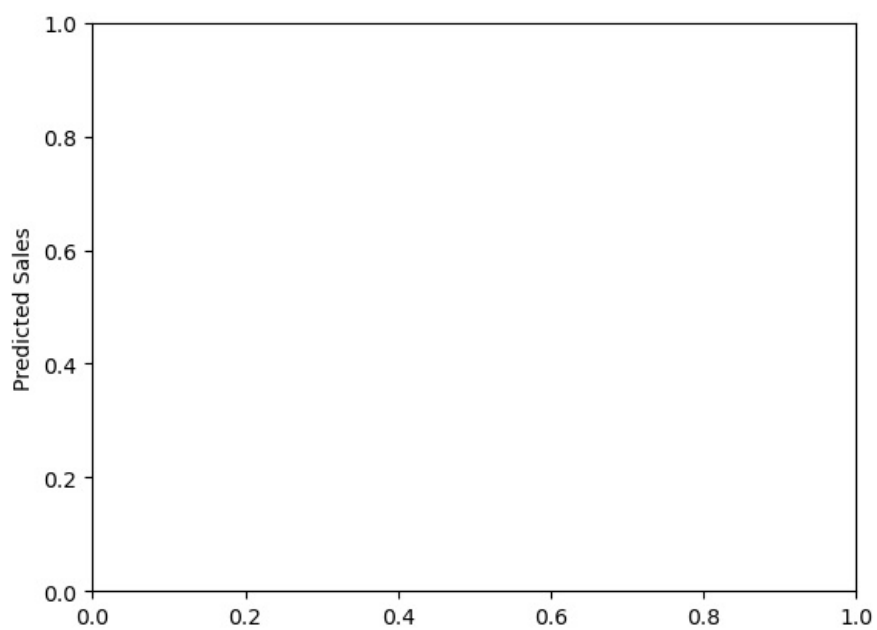
```
In [31]: plt.xlabel("Actual Sales")
```

```
Out[31]: Text(0.5, 0, 'Actual Sales')
```



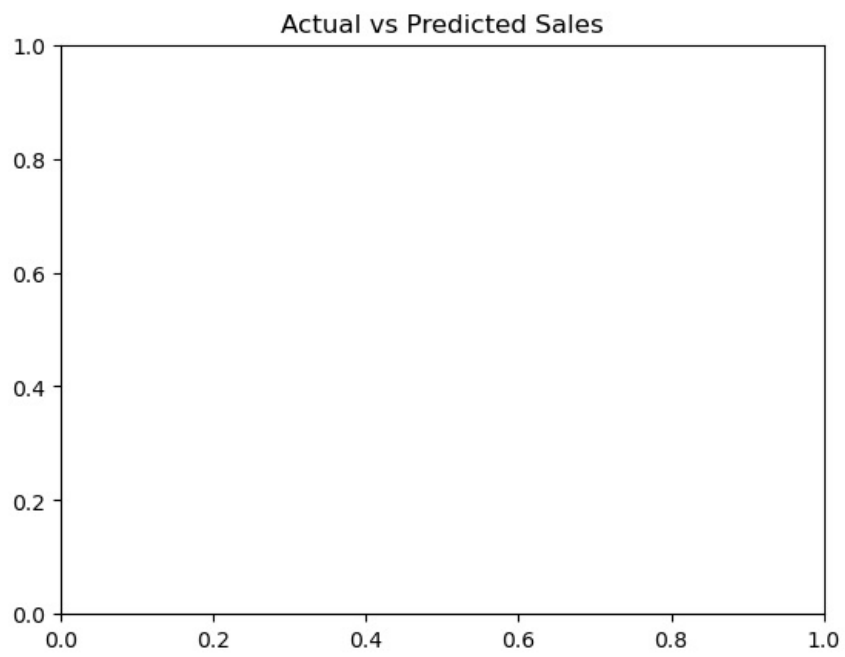
```
In [32]: plt.ylabel("Predicted Sales")
```

```
Out[32]: Text(0, 0.5, 'Predicted Sales')
```



```
In [33]: plt.title("Actual vs Predicted Sales")
```

```
Out[33]: Text(0.5, 1.0, 'Actual vs Predicted Sales')
```



```
In [34]: plt.show()
```

```
In [ ]:
```